percentages of lead and tin.

The test set should look like the following PyTorch tensor.

```
tensor([[0.0000, 0.0000, 0.0000, 0.0000],
        [0.0000, 0.0100, 0.0000, 0.0000],
        [0.0000, 0.0200, 0.0000, 0.0000],
        ...,
        [1.0000, 0.9800, 0.0000, 0.0000],
        [1.0000, 0.9900, 0.0000, 0.0000],
        [1.0000, 1.0000, 0.0000, 0.0000]])
```

Notice that the third and forth columns are all zeros.

5. Predict the mixing temperature on this test set. That is, compute the posterior mean and standard deviation of the normalized mixing temperature for every point in the test set.
6. Visualize the predictions. This involves showing the mean and standard deviation as heat maps.

The solution is included in `CH02/02 - Exercise.ipynb`, which is explained as follows.

1. We first import the necessary libraries: PyTorch for array/tensor manipulation, GPyTorch for GP modeling, and Matplotlib for visualization.

```
import torch
import gpytorch
import matplotlib.pyplot as plt
```

We then store the numbers in Table A.1 in two PyTorch tensors, `train_x` and `train_y`, which respectively contain the features and labels of our data set.

```
train_x = torch.tensor(
    [
        [1 / 2, 1 / 2, 0, 0],
        [1 / 3, 1 / 3, 1 / 3, 0],
        [0, 1 / 2, 1 / 2, 0],
        [0, 1 / 3, 1 / 3, 1 / 3],
    ]
)

train_y = torch.tensor([192.08, 258.30, 187.24, 188.54])
```

2. We normalize the labels as follows.

```
# normalize the labels
train_y = (train_y - train_y.mean()) / train_y.std()
```

When printed out, `train_y` should contain the following values: `tensor([-0.4183, 1.4974, -0.5583, -0.5207])`.

3. We reimplement our GP model class as follows.

```
class BaseGPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ZeroMean()
        self.covar_module = gpytorch.kernels.RBFKernel()

    def forward(self, x):
        mean_x = self.mean_module(x)
        covar_x = self.covar_module(x)
        return gpytorch.distributions.MultivariateNormal(mean_x, covar_x)
```

We then initialize an object of this class with our training data.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

**Licensed to ashwin sampath <ashwin.manning@gmail.com>**